# akeyless-watchover

## Technical reference

Skill commands · kit commands · options · examples

Version 1.7.0

October 29, 2026

https://github.com/MisterErre/akeyless-watchover

# Contents


**1 Overview** **1**

1.1 Command index . . . 1

**2 Set-up** **2**

2.1 Kit . . . 2

2.2 Skill, for any AI agent . . . 2

**3 Skill commands** **3**

3.1 `help` . . . 3

3.2 `design` . . . 3

3.3 `change` . . . 4

3.4 `review` . . . 4

3.5 `pipeline` . . . 4

3.6 `import` . . . 5

3.7 `reconcile` . . . 5

3.8 `conjur` . . . 5

3.9 `pki` . . . 6

3.10 `harden` . . . 6

3.11 `gateway` . . . 6

3.12 `verify` . . . 7

3.13 `cli` . . . 7

3.14 `explain` . . . 7

**4 Kit commands** **9**

4.1 `test` . . . 9

4.2 `simulate` . . . 9

4.3 `verify` . . . 10

4.4 `export` . . . 11

4.5 `reconcile` . . . 13

4.6 `forwarder` . . . 14

4.7 `conjur` . . . 16

4.8 `account-settings` . . . 16

4.9 `token` . . . 17

4.10 `build-skill` . . . 18

4.11 `install-skill` . . . 18

4.12 `validate-skill` . . . 19

4.13 `mcp` . . . 19

4.14 `help` and `version` . . . 20

**A Model files** **21**

**B Guard messages** **22**

**C Variables** **23**

C.1 Environment . . . . . 23

C.2 Terraform input variables . . . . . 23

**D Files** **24**

# 1 Overview

akeyless-watchover manages the governance of an Akeyless account as code: paths, access roles, auth methods and sub-claims, account settings, PKI, event forwarders and Gateway administration, modelled in YAML and delivered by Terraform from one repository. It has two interfaces:

| | **Skill commands** (Chapter 3) | **Kit commands** (Chapter 4) |
|---|---|---|
| Used by | an AI agent (any agent and model) | you, CI, or an agent with a shell / MCP |
| Syntax | `akeyless-watchover <command> [request]` | `scripts/watchover <command> [options]` |
| Produces | designs, model diffs, reviews, exact commands | tests, simulations, exports, scaffolds, packages |
| Runs where | in the conversation | in the governance repository |

**Notation.** `<x>` required, `[x]` optional, `a|b` choice; *repeatable* options may be given several times. *Live* commands call Akeyless; all others are offline. **VERIFY** marks a fact no primary source confirms. Exit codes: 0 success, 1 check failed, 2 usage or execution error (per command: each entry).

## 1.1 Command index


| **Skill command** | | **p.** |
|---|---|---|
| `help` | command reference | 3 |
| `design` | diagram → model | 3 |
| `change` | minimal access change | 4 |
| `review` | review diff or plan | 4 |
| `pipeline` | repository + CI set-up | 4 |
| `import` | existing config → Terraform | 5 |
| `reconcile` | repository vs Akeyless | 5 |
| `conjur` | Conjur migration | 5 |
| `pki` | CA issuers, requesters | 6 |
| `harden` | account hardening | 6 |
| `gateway` | Gateway admin, log forwarding | 6 |
| `verify` | who can access what | 7 |
| `cli` | exact CLI commands | 7 |
| `explain` | concepts, verified facts | 7 |

| **Kit command** | | | **p.** |
|---|---|---|---|
| `test` | all offline checks | | 9 |
| `simulate` | expectations vs model | | 9 |
| `verify` | expectations vs account | live | 10 |
| `export` | account → model / HCL | live | 11 |
| `reconcile` | repository vs Akeyless | live | 13 |
| `forwarder` | event forwarder scaffold | | 14 |
| `conjur` | Conjur policy → roles | | 16 |
| `account-settings` | CLI-only settings | live | 16 |
| `token` | CI JWT → token | live | 17 |
| `build-skill` | skill package, prompt pack | | 18 |
| `install-skill` | install for an agent | | 18 |
| `validate-skill` | check against the spec | | 19 |
| `mcp` | MCP server | | 19 |
| `help, version` | reference, version | | 20 |

# 2 Set-up

## 2.1 Kit

Requirements: Python 3.10+ with `pyyaml` (and `pytest` for `test`); Terraform $\geq$ 1.7 or OpenTofu $\geq$ 1.8; for live commands the `akeyless` CLI or `curl`+`jq` (`token`). Copy `terraform/`, `bootstrap/`, `scripts/` and the CI template from `ci/` into your governance repository (GitHub, GitHub Enterprise or GitLab); check with `scripts/watchover test`. GitHub Enterprise Server: bootstrap with `ci.github_url` (OIDC issuer `https://<host>/_services/token`, JWKS reachable from Akeyless) and set `RUNNER_LABEL` for self-hosted runners. Agents never commit or push to this repository: they deliver files and patches (`reconcile`, 4.5).

## 2.2 Skill, for any AI agent

| Agent supports | Do |
|---|---|
| Agent Skills (Claude, Codex, Copilot, Cursor, Gemini CLI, …) | `scripts/watchover install-skill --agent <agent>` (4.11); in apps with a skills screen upload `dist/akeyless-watchover.zip` |
| MCP (stdio) | register `python3 <kit>/scripts/watchover_mcp.py --repo <governance repo>` (4.13) |
| neither | prompt pack from `build-skill` (4.10): `system-prompt.md` or `system-prompt-short.md` as instructions, `knowledge/` as files |

Restart the agent after installing; test with `akeyless-watchover help`. Folders per agent: `install-skill` options; client snippets: `docs/agents.md`.

```
# MCP registration examples (replace the paths)
claude mcp add watchover -- python3 /kit/scripts/watchover_mcp.py --repo /governance
gemini mcp add watchover python3 /kit/scripts/watchover_mcp.py --repo /governance
# Codex ~/.codex/config.toml:  [mcp_servers.watchover] command="python3" args=["/kit/scripts/watchover_mcp.py","--
    repo","/governance"]
# VS Code .vscode/mcp.json "servers" / Cursor .cursor/mcp.json and Claude Desktop "mcpServers": same command and
    args
```

# 3 Skill commands

Typed to an AI agent that has the skill (Chapter 2). Syntax: `akeyless-watchover <command> [args]`, the agent's own skill syntax, or the MCP prompt `watchover-<command>`. Arguments are free text; a plain-language request is mapped to the closest command. Every command ends with the verification that applies and the VERIFY caveats that touch the answer. Example answers are abridged and vary with the agent and model.

## 3.1 `help`

**Syntax** `akeyless-watchover help [command]`

**Does** Show this guide, or the details of one skill command.

**Delivers** The command table, parameters and examples.

**How**
- Reads references/usage-guide.md (or the command table in SKILL.md).
- With an argument, returns only that command's entry. Parameters are never invented.

**Example**

**You** akeyless-watchover help change
**Agent** Shows the `change` entry: arguments (*an access request in plain words*), what it delivers (a minimal diff of `model/roles/<bu>.yaml`, new expectations, who must approve) and an example. Without an argument: the table of all skill commands and the kit commands.

## 3.2 `design`

**Syntax** `akeyless-watchover design <diagram | description of BUs, apps, environments, personas>`

**Does** Translate an organisation/structure diagram into Akeyless configuration.

**Delivers** Path tree, access matrix, YAML model (roles, auth methods, folders), access expectations, plan expectations, CLI equivalent.

**How**
- Inventories BUs, applications, environments, data classes, personas and the identity source of each; asks for whatever is missing.
- Derives the path tree `/<bu>/<app>/<env>/...` (delegation boundary first, carve-outs such as `break-glass` as fixed segments) and the access matrix persona x scope.
- Creates one role per persona x scope x access level; binds humans through the SSO auth method + `groups` sub-claims and workloads through one auth method per trust boundary.
- Writes the model files and access_expectations.yaml, then runs `watchover simulate/test` when it can run commands.

**Example**

**You** akeyless-watchover design *[attached diagram: BU Payments with apps checkout, refunds; prod and nonprod; Entra groups; EKS workloads]*
**Agent** **1. Path tree** `/payments/{checkout,refunds}/{prod,nonprod}/...`, `/payments/<app>/prod/break-glass/...`
**2. Access matrix** payments-admin: nonprod CRUD, prod read; checkout-prod-pod: own prod read; auditor: list everything; everyone else: nothing.
**3. Model** `model/roles/payments.yaml` (`team-admin`, `checkout-prod-workload`, ...), `model/auth_methods.yaml` (OIDC with `force_sub_claims`, K8s bound to namespace `payments`), `model/folders.yaml`.
**4. Expectations** 11 checks including the negative ones (break-glass, other BUs).
**5. Plan expectations and CLI** "+ 2 roles, + 2 auth methods, + 3 associations, no replacements"; the equivalent `akeyless create-role / set-role-rule / assoc-role-am` commands.

## 3.3 change

**Syntax** `akeyless-watchover change <access request in plain words>`

**Does** Change existing access with the smallest possible model diff.

**Delivers** Diff of model/roles/<bu>.yaml (and auth_methods.yaml if needed), new expectations, who must approve (CODEOWNERS).

**How**
- Finds the role file of the owning business unit (model/roles/<bu>.yaml).
- Applies the smallest change: a rule, an association, narrower sub-claims - or a new role when no existing role has the right audience.
- Adds a positive and a negative expectation, simulates, and lists the CODEOWNERS approvers.

**Example**

**You** akeyless-watchover change give the fraud-analysts Entra group read on refunds prod
**Agent** Smallest diff – a new role, because no existing role has the right audience:

```
+  /governance/roles/payments/fraud-analysts:
+    description: Fraud analysts read refunds prod
+    rules:
+      - { path: "/payments/refunds/prod/*", capabilities: [read, list] }
+      - { path: "/payments/refunds/prod/break-glass/*", capabilities: [deny] }
+    associations:
+      - auth_method: /governance/auth/sso-entra-oidc
+        sub_claims: { groups: [grp-fraud-analysts] }
```

New expectations (one positive, two negative), the expected plan (+1 role, +1 association) and the approvers from CODEOWNERS: payments owners and the platform team.

## 3.4 review

**Syntax** `akeyless-watchover review <model files, merge-request diff or plan output>`

**Does** Review a proposed change for least privilege, guard violations and blast radius.

**Delivers** Findings by severity, failing guards, missing expectations, plan red flags (replacements).

**How**
- Checks the input against the guards, least privilege (root scope, removed deny rules, associations without sub-claims) and plan red flags (replacements caused by renames).
- Reports findings ranked by severity, with the missing expectations.

**Example**

**You** akeyless-watchover review *[pasted plan]*
**Agent** **High**: `akeyless_auth_method_k8s.this["/governance/auth/k8s-prod"]` *must be replaced* – the name changed; consumers lose their access ID. Revert the rename or plan a cut-over. **Medium**: rule `/payments/*` with `update` on `team-admin` widens access to every app's prod; no expectation covers it. **OK**: 2 in-place rule updates match the request.

## 3.5 pipeline

**Syntax** `akeyless-watchover pipeline [github | gitlab] [repository]`

**Does** Set up the governance repository: bootstrap identity, CI, CODEOWNERS, state.

**Delivers** bootstrap.tfvars, CI file, variables to create, step-by-step order.

**How**
- Fills bootstrap.tfvars for the CI platform and repository, selects the CI template, lists the CI variables (masked or not) and the order of the steps.

**Example**

**You** akeyless-watchover pipeline gitlab acme/platform/akeyless-governance
**Agent** `bootstrap.tfvars` (`platform = "gitlab"`, `repository = "acme/platform/akeyless-governance"`, `gitlab_url`), the order of steps (bootstrap as admin → CI variables → copy `ci/gitlab/`.

`gitlab-ci.yml` → protected branch and environment → CODEOWNERS), the list of CI variables to create and which ones must be masked.

## 3.6 `import`

**Syntax** `akeyless-watchover import [what to export: everything / a type / one user's view] [export directory or REPORT.md]`

**Does** Export existing config (RBAC, auth methods incl. OIDC, folders, custom fields, key policies, PKI issuers, account settings) and translate it into Terraform.

**Delivers** The exact `watchover export` command (model or --hcl mode, filters), then resolution of every REPORT.md item and the import checklist until the plan shows imports only.

**How**

- Chooses the export mode (model or --hcl) and filters, and gives the `watchover export` command.
- Reads REPORT.md and resolves every item (secrets, unsupported types, external references).
- Iterates until the plan shows imports only.

**Example**

**You** akeyless-watchover import the OIDC auth method and everything alice@example.com can reach
**Agent** The command to run:

```
scripts/watchover export --out export/ --profile admin --types roles,auth-methods \
  --persona-auth-method /governance/auth/sso-entra-oidc \
  --persona-claim email=alice@example.com --persona-claim groups=grp-akl-payments-admins
```

Then, given `export/REPORT.md`: one decision per report item (client secret into `TF_VAR_auth_method_secrets`, a temporary rule that will not be imported, an association to an unmanaged auth method), where to move each file, and "iterate until the plan shows imports only".

## 3.7 `reconcile`

**Syntax** `akeyless-watchover reconcile [repository URL or checkout] [branch]`

**Does** Compare the governance repository (GitHub Enterprise) with live Akeyless and prepare the changes for manual review. Never commits or pushes.

**Delivers** REPORT.md with every difference and its two resolutions, changes.patch, proposed files, imports.tf - to review and commit yourself.

**How**

- Clones the repository read-only (or reads a checkout) and runs `watchover reconcile` with a read-only Akeyless identity; without a shell, gives the command to run.
- Explains each difference and asks whether Akeyless or the repository is right; adjusts the proposal (--only, --include-unmodelled, --propose-removals).
- Hands over the patch and files with the commands to apply and verify them (git apply, terraform plan). Never commits, pushes or opens a pull request.

**Example**

**You** akeyless-watchover reconcile https://ghe.example.com/platform/akeyless-governance main
**Agent** Ran `watchover reconcile` with the read-only profile: 2 objects changed outside Git, 1 only in Akeyless, 1 only in the repository. (1) `payments/team-admin` gained `create` on `/payments/+/nonprod/*` in the console – intended? Keep the hunk to codify it, drop it to let the next apply revert it. (2) … The patch, the complete files and `imports.tf` are in `reconcile/`; apply on a branch with `git apply --3way reconcile/changes.patch`, run `terraform plan` (imports and kept changes only), then commit through a pull request. I have not committed or pushed anything.

## 3.8 `conjur`

**Syntax** `akeyless-watchover conjur <Conjur policy YAML>`

**Does** Migrate from CyberArk Conjur: secrets via the Gateway, RBAC via translation.

**Delivers** Translated roles, bindings to fill (auth method + sub-claims), migration resource, cut-over plan.

**How**
- Runs or gives `watchover conjur`; proposes an auth method and sub-claims for each Conjur host and user in bindings.yaml; adds the Gateway migration entry and a cut-over plan.

**Example**

**You** akeyless-watchover conjur *[attached root.yml and prod.yml]*
**Agent** Runs (or tells you to run) `scripts/watchover conjur root.yml prod.yml --out export/conjur`, explains each translated role (Conjur layer/group → Akeyless role on the migrated paths), and for every entry of `bindings.yaml` proposes the auth method and sub-claims (e.g. host `prod/payments/checkout` → K8s auth, namespace `payments`). Plus the Gateway migration entry for `model/gateway.yaml` and a cut-over plan.

## 3.9 `pki`

**Syntax** `akeyless-watchover pki <certificate requirements>`

**Does** Design private/public CA issuers and who may request certificates.

**Delivers** pki.yaml (CA keys, issuers), requester roles, expiry forwarder, CLI to issue/revoke.

**How**
- Writes pki.yaml (CA key, issuers: signer key or public CA target, domains, TTL, auto-renew), the requester roles, an expiry forwarder and the CLI to issue and revoke.

**Example**

**You** akeyless-watchover pki internal TLS for *.payments.internal, 90 days, auto-renew
**Agent** `model/pki.yaml` with a CA key (`/pki/ca/internal-root`) and an issuer (`/pki/issuers/payments-server`, `allowed_domains`, TTL 7776000 s, `auto_renew` with `destination_path`); a requester role for the payments workloads; an expiry forwarder (`watchover forwarder --preset certificates`); the CLI to issue and revoke a certificate.

## 3.10 `harden`

**Syntax** `akeyless-watchover harden [current settings or concerns]`

**Does** Account-level hardening.

**Delivers** governance.yaml: account_settings (Terraform), account_settings_cli (CLI), custom fields, key policies, event forwarders.

**How**
- Splits the settings into the Terraform-managed subset (account_settings) and the CLI-only part (account_settings_cli); adds custom fields, key policies and critical event forwarders.
- Always starts with `watchover account-settings --dry-run`.

**Example**

**You** akeyless-watchover harden
**Agent** Proposals with reasons, split by where they are delivered: `account_settings` (Terraform: password policy, max versions, JWT TTLs, item deletion protection), `account_settings_cli` (bound IPs, allowed client types, locks – applied by `watchover account-settings`; *dry-run first*), mandatory custom fields such as `owner`, key policies, and critical event forwarders.

## 3.11 `gateway`

**Syntax** `akeyless-watchover gateway <who administers which Gateway, SIEM target>`

**Does** Gateway admin plane and audit-log forwarding.

**Delivers** gateway.yaml (allowed_access, log_forwarding), gateway_api setup.

**How**
- Writes gateway.yaml: allowed_access (auth method or access ID + permissions) and log forwarding (syslog or splunk); sets gateway_api. One state per Gateway cluster.

**Example**

**You** akeyless-watchover gateway platform admins full, PKI team targets only, Splunk
**Agent** `model/gateway.yaml`: `allowed_access` entries (platform admins through the SSO auth method with their group sub-claim; PKI team limited to the targets permission), `log_forwarding` of type `splunk` with `splunk_token_var`, and the reminder to set `gateway_api` – one state per Gateway cluster.

## 3.12 `verify`

**Syntax** `akeyless-watchover verify [persona | path]`

**Does** Prove who can access what.

**Delivers** access_expectations.yaml checks, `watchover simulate` / `watchover verify` commands, Impersonate As steps.

**How**
- Evaluates access with the Akeyless rules (wildcards, deny precedence, sub-claims OR within a key / AND across keys).
- Writes the matching expectation checks and the commands that prove them offline (`simulate`) and live (`verify`, Impersonate As).

**Example**

**You** akeyless-watchover verify can the checkout pods read refunds prod?
**Agent** **No.** `checkout-prod-workload` only matches `/payments/checkout/prod/*`; no other role is associated with the K8s auth method. Adds the negative check `{ persona: checkout-prod-pod, path: /payments/refunds/prod/db, expect: [] }` and shows how to prove it offline (`watchover simulate --explain`) and live (`watchover verify`, or *Impersonate As* in the console).

## 3.13 `cli`

**Syntax** `akeyless-watchover cli <task>`

**Does** Exact Akeyless CLI commands for a task (break-glass, bootstrap, one-off checks).

**Delivers** Commands checked against the CLI reference, with the equivalent model change.

**How**
- Gives the exact `akeyless` commands, checked against the CLI reference, and the model change that makes the result permanent (CLI changes to roles are reverted by the next apply).

**Example**

**You** akeyless-watchover cli create a temporary read rule for 60 minutes
**Agent**

```
akeyless set-role-rule --role-name /governance/roles/payments/team-admin \
  --path "/payments/refunds/prod/*" --capability read --capability list --ttl 60
```

With the caveat that temporary rules are CLI/API-only and the next apply on that role removes them – and the model change to make if the access should last.

## 3.14 `explain`

**Syntax** `akeyless-watchover explain <topic>`

**Does** Explain an Akeyless concept precisely (sub-claims, deny, wildcards, anti-escalation...).

**Delivers** A short answer with the verified facts and VERIFY caveats.

**How**
- Answers from the verified references; anything not confirmed by a primary source is marked VERIFY.

**Example**

**You** akeyless-watchover explain how do sub-claims combine?
**Agent** Within one key the listed values are alternatives (OR); across keys all must match (AND). With `force_sub_claims` on the auth method, an association without sub-claims is rejected – the kit's guard enforces it before Akeyless does. Glob patterns in values are supported; case sensitivity is per association.

# 4 Kit commands

Run in the governance repository (or `template/` of an installed skill): `scripts/watchover <command> [options]`; `<command> --help` or `help <command>` prints the entry in the terminal. *Live* commands call Akeyless; the others are offline. Each entry ends with a real execution (Section D).

## 4.1 `test`

**Synopsis** `scripts/watchover test [options]` *offline*

**Does** Run every offline check (what CI runs).

**How**
- Runs, stopping at the first failure: fmt -check, init -backend=false, validate and `test` (mocked provider) in terraform/ and bootstrap/; guard_messages.sh (each negative fixture fails with the intended guard); export_roundtrip.sh; `simulate`; pytest.

**Exit codes** 0 all passed; otherwise the exit code of the first failing step

| Option | Value | Required / default | Description |
|---|---|---|---|
| `--tool` | `terraform\|tofu` | default terraform | IaC binary to use |
| `--skip-iac` | – | optional | only run the Python checks |

**Examples.**

```
scripts/watchover test
scripts/watchover test --tool tofu
```

**Sample run.**

```
$ scripts/watchover test --skip-iac

$ /usr/local/bin/python3 ./scripts/akl_simulate.py --model terraform/model --expectations
↪  terraform/model/access_expectations.yaml   (in .)
[... 14 lines omitted ...]
$ /usr/local/bin/python3 -m pytest -q scripts/tests   (in .)
.......................................................................................... [ 84%]
.................                                                                        [100%]
109 passed in <t>s

all checks passed
```

exit code 0

## 4.2 `simulate`

**Synopsis** `scripts/watchover simulate [options]` *offline*

**Does** Offline: evaluate access_expectations.yaml against the model.

**How**
- Loads model/roles/*.yaml and auth_methods.yaml.
- For each check: selects the roles whose associations match the persona's auth method and claims; collects the rules of the right type whose path pattern matches (claim templates expanded); a matching deny removes everything.
- Compares the resulting capability set with `expect` (exact match).

**Exit codes** 0 all met; 1 mismatch; 2 model error

| Option | Value | Required / default | Description |
|---|---|---|---|
| `--model` | `MODEL` | **required** | model directory (terraform/model) |
| `--expectations` | `EXPECTATIONS` | **required** | access_expectations.yaml |
| `--explain` | – | optional | print the matching rules for every check, not only failures |

**Examples.**

```
scripts/watchover simulate --model terraform/model --expectations terraform/model/access_expectations.yaml --explain
```

**Sample run.**

```
$ scripts/watchover simulate --model terraform/model --expectations terraform/model/access_expectations.yaml --explain
PASS  payments-admin          item        /payments/checkout/nonprod/db/app                expect=['create', 'delete', 'list',
↪  'read', 'update'] got=['create', 'delete', 'list', 'read', 'update']
        /governance/roles/payments/team-admin: /payments/+/nonprod/* -> ['create', 'delete', 'list', 'read', 'update']
PASS  payments-admin          item        /payments/checkout/prod/db/app                   expect=['list', 'read']
↪  got=['list', 'read']
        /governance/roles/payments/team-admin: /payments/+/prod/* -> ['list', 'read']
PASS  payments-admin          item        /payments/checkout/prod/break-glass/root         expect=[] got=[]
        /governance/roles/payments/team-admin: /payments/+/prod/* -> ['list', 'read']
        /governance/roles/payments/team-admin: /payments/+/prod/break-glass/* -> ['deny']
[... 18 lines omitted ...]
        /governance/roles/platform/pki-requesters-payments: /pki/issuers/payments-server -> ['list', 'read']

0 mismatch(es)

                                                                                exit code 0
```

## 4.3 `verify`

**Synopsis** `scripts/watchover verify [options]` *live*

**Does** Live: compare expectations with describe-permissions run as each persona's CLI profile.

**How**
- For each check whose persona has a `profile`: runs `akeyless describe-permissions --path --type --json --profile <persona>` and compares `client_permissions` with `expect`.
- Personas without a profile are skipped and counted.

**Exit codes** 0 all met; 1 mismatch; 2 CLI call failed

| Option | Value | Required / default | Description |
|---|---|---|---|
| `--expectations` | `EXPECTATIONS` | **required** | access_expectations.yaml (personas need a 'profile') |
| `--akeyless` | `AKEYLESS` | default akeyless | path to the akeyless CLI binary |
| `--error-means-no-access` | – | optional | treat a failing describe-permissions call as 'no permissions' (check the CLI behaviour first) |

**Examples.**

```
scripts/watchover verify --expectations terraform/model/access_expectations.yaml
```

**Sample run.** The stand-in account has one console change that is not in Git:

```
$ scripts/watchover verify --expectations terraform/model/access_expectations.yaml
PASS  payments-admin          /payments/checkout/nonprod/db/app                expect=['create', 'delete', 'list', 'read',
↪  'update'] got=['create', 'delete', 'list', 'read', 'update']
PASS  payments-admin          /payments/checkout/prod/db/app                   expect=['list', 'read'] got=['list', 'read']
PASS  payments-admin          /payments/checkout/prod/break-glass/root         expect=[] got=[]
PASS  payments-admin          /hr/payroll/prod/db                              expect=[] got=[]
PASS  auditor                 /payments/checkout/prod/db/app                   expect=['list'] got=['list']
[... 1 lines omitted ...]
FAIL  random-employee         /payments/checkout/prod/db/app                   expect=[] got=['list', 'read']
PASS  checkout-prod-pod       /payments/checkout/prod/db/app                   expect=['list', 'read'] got=['list', 'read']
```

```
PASS  checkout-prod-pod        /payments/checkout/prod/break-glass/root          expect=[] got=[]
PASS  checkout-prod-pod        /payments/checkout/nonprod/db/app                 expect=[] got=[]
PASS  checkout-prod-pod        /pki/issuers/payments-server                      expect=['list', 'read'] got=['list', 'read']

1 mismatch(es), 0 check(s) skipped (persona without profile)
```

exit code 1

## 4.4 `export`

**Synopsis** `scripts/watchover export [options]` *live*

**Does** Export existing config and translate it: YAML model + import blocks, or plain Terraform (--hcl).

**How**

- Reads the selected object families through the CLI (`--json`) or the REST API (token); filters by name, root folder, depth or persona.
- Pagination: the CLI returns a `next_page` token even after the last page, so reading stops at the first EMPTY page; also when the token is missing or repeats, or a page brings nothing new; it aborts after 10,000 pages instead of looping.
- Writes raw/ (JSON, secrets redacted) and REPORT.md (items needing a decision).
- Model mode: model/*.imported.yaml + imports.tf for the kit.
- --hcl: complete resources laid out like raw/ - raw/<dir>/<name>.json becomes hcl/<dir>/<name>.tf (each directory a module), root JSON files become root .tf files; hcl/main.tf calls the modules, hcl/imports.tf adopts every object.
- Attributes follow the provider's own read code (scripts/data/provider-2.0.5.json): API field -> attribute, converted to the schema type. Secrets become var.secrets[...], required values the API does not return var.inputs[...] (terraform.tfvars.example); a field whose name is an attribute but that the read code does not map is set with a VERIFY comment; untranslated fields stay as `# raw` comments.
- --hcl also reads details the provider reads (target-get-details, get-group, Gateway defaults/cache/remote access/log forwarding/migrations/allowed access); --no-details skips them. --generate adds hcl-generated/ where Terraform writes its own resources, for comparison.

**Exit codes** 0 written; 2 read/HTTP error or invalid options

| Option | Value | Required / default | Description |
|---|---|---|---|
| `--out` | `OUT` | **required** | output directory (model/, imports.tf or hcl/, raw/, REPORT.md) |
| `--transport` | `cli\|api` | default cli | how to read Akeyless: the akeyless CLI (profiles, SSO logins) or the REST API directly (token) |
| `--profile` | `PROFILE` | optional | cli transport: akeyless CLI profile with read access to the objects to export |
| `--akeyless` | `AKEYLESS` | default akeyless | cli transport: path to the akeyless CLI binary |
| `--gateway-url` | `GATEWAY_URL` | optional | cli transport, --types gateway: Gateway configuration URL (https://gw:8000) |
| `--api` | `API` | default https://api.akeyless.io | api transport: API base URL (default $AKEYLESS_API or https://api.akeyless.io) |
| `--token` | `TOKEN` | optional | api transport: Akeyless token (default $AKEYLESS_TOKEN; e.g. from 'watchover token') |
| `--gateway-api` | `GATEWAY_API` | optional | api transport, --types gateway: Gateway V2 API (https://gw:8000/api/v2) |
| `--from-dir` | `FROM_DIR` | optional | replay a raw/ directory of a previous export instead of calling the CLI |

| Option | Value | Required / default | Description |
|---|---|---|---|
| `--types` | `TYPES` | default roles,auth-methods,folders,custom-fields,key-policies,pki-issuers,account-settings,targets,groups | comma-separated object families (default: all but gateway): roles, auth-methods, folders, custom-fields, key-policies, pki-issuers, account-settings, targets, groups, gateway |
| `--only` | `ONLY` | optional | keep only objects whose name matches this regex |
| `--exclude` | `EXCLUDE` | *repeatable* | regex of names to skip (repeatable) |
| `--root` | `ROOT` | default / | folder to start the item/folder walk from |
| `--max-depth` | `MAX_DEPTH` | default 20 | maximum folder depth of the walk |
| `--persona-auth-method` | `PERSONA_AUTH_METHOD` | optional | keep one identity's view: its auth method ... |
| `--persona-claim` | `PERSONA_CLAIM` | *repeatable* | ... and its claims, key=value (repeatable); keeps roles whose association matches |
| `--hcl` | – | optional | write complete Terraform resources laid out like raw/ (hcl/: one module per directory) plus import blocks |
| `--no-details` | – | optional | --hcl: skip the detail reads (target details, groups, Gateway parts); fewer attributes |
| `--generate` | – | optional | with --hcl: also write hcl-generated/ (flat import blocks) and run 'terraform plan -generate-config-out' there, to compare Terraform's own output with the translation (needs terraform and AKEYLESS_ACCESS_ID/AKEYLESS_ACCESS_KEY in the environment) |
| `--terraform` | `TERRAFORM` | default terraform | terraform or tofu binary used by --generate |

**Examples.**

```
scripts/watchover export --out export/ --profile admin --exclude '^/governance/auth/ci-terraform$'
scripts/watchover export --out export/ --profile admin --types auth-methods --only sso
scripts/watchover export --out export/ --profile admin --persona-auth-method /governance/auth/sso-entra-oidc
↪  --persona-claim email=alice@example.com --persona-claim groups=grp-akl-payments-admins
AKEYLESS_ACCESS_ID=p-... AKEYLESS_ACCESS_KEY=... scripts/watchover export --out export/ --profile admin --hcl
↪  --generate
AKEYLESS_TOKEN=$(scripts/watchover token) scripts/watchover export --out export/ --transport api --hcl --types
↪  targets,groups,gateway --gateway-api https://gw.example.com:8000/api/v2
scripts/watchover export --out export/ --profile admin --hcl --no-details --types roles,auth-methods
scripts/watchover export --out export2/ --from-dir export/raw     # offline replay
```

**Sample run.** HCL mode: the same layout as the JSON, complete resources, the secret as a variable:

```
$ scripts/watchover export --out export/ --profile admin --hcl --types roles,auth-methods,targets && find export/raw/roles
↪  export/hcl/roles -type f | sort && sed -n '2,15p' export/hcl/auth_methods/governance_auth_sso.tf
exported (hcl): 3 roles, 7 auth-methods, 3 targets; 9 item(s) in REPORT.md
export/hcl/roles/governance_roles_payments_team-admin.tf
export/hcl/roles/governance_roles_platform_auditors.tf
export/hcl/roles/governance_roles_platform_god.tf
export/hcl/roles/versions.tf
export/raw/roles/governance_roles_payments_team-admin.json
export/raw/roles/governance_roles_platform_auditors.json
export/raw/roles/governance_roles_platform_god.json
# matched by field name, not by the provider read code (VERIFY): delete_protection
# raw auth_method_access_id = "p-sso"
resource "akeyless_auth_method_oidc" "governance_auth_sso" {
  name                = "/governance/auth/sso"
  allowed_redirect_uri = ["https://console.akeyless.io"]
  audit_logs_claims    = ["email"]
  bound_ips            = ["10.0.0.0/8", "192.168.0.0/16"]
  client_id            = "cid"
  client_secret        = var.secrets["/governance/auth/sso:client_secret"]
  delete_protection    = "true"
```

```
description          = "Entra SSO"
force_sub_claims     = true
issuer               = "https://login.microsoftonline.com/t/v2.0"
jwt_ttl              = 60
```

exit code 0

## 4.5 `reconcile`

**Synopsis** `scripts/watchover reconcile [options]` *live*

**Does** Compare the governance repository (e.g. GitHub Enterprise) with live Akeyless; write a report and a patch to review. Read-only: never commits or pushes.

**How**

- Gets the repository read-only: shallow clone of --ref with the push URL disabled, or reads --repo-dir as it is. Never writes to it, never commits or pushes.
- Exports roles, auth methods, folders, custom fields and key policies from Akeyless (as `export`; or uses --export-dir / --from-dir).
- Compares object by object after normalisation (defaults, ordering, *_var secret keys and guard switches ignored): changed, only in Akeyless, only in the repository.
- Writes REPORT.md, changes.patch (`git apply`), proposed/ (complete changed files at their repository paths, edited in place so comments survive), imports.tf for objects added from Akeyless, summary.json and the export used.

**Exit codes** 0 no differences; 1 differences found (files written); 2 git, export or model error

**Environment** `AKEYLESS_TOKEN`: api transport: token when --token is not given
`AKEYLESS_API`: api transport: API URL

| Option | Value | Required / default | Description |
|---|---|---|---|
| `--repo-url` | `REPO_URL` | optional | governance repository to clone read-only (GitHub Enterprise HTTPS or SSH URL; uses your git credentials, e.g. `gh auth login --hostname <ghe-host>`) |
| `--repo-dir` | `REPO_DIR` | optional | existing local checkout to compare (read only, never modified) |
| `--ref` | `REF` | default main | branch to compare with --repo-url |
| `--model-path` | `MODEL_PATH` | default terraform/model | model directory inside the repository |
| `--out` | `OUT` | default reconcile | output directory (replaced; keep it out of Git) |
| `--export-dir` | `EXPORT_DIR` | optional | use an earlier `watchover export` output instead of exporting now |
| `--from-dir` | `FROM_DIR` | optional | export offline from a raw/ directory (as `export --from-dir`) |
| `--transport` | `cli\|api` | default cli | how to read Akeyless (as `export`) |
| `--profile` | `PROFILE` | optional | cli transport: akeyless CLI profile (a read-only identity is enough) |
| `--akeyless` | `AKEYLESS` | default akeyless | cli transport: path to the akeyless CLI binary |
| `--api` | `API` | optional | api transport: API URL (default $AKEYLESS_API or https://api.akeyless.io) |
| `--token` | `TOKEN` | optional | api transport: Akeyless token (default $AKEYLESS_TOKEN) |
| `--only` | `ONLY` | optional | compare only objects whose name matches this regex |
| `--exclude` | `EXCLUDE` | *repeatable* | regex of names to skip (repeatable) |
| `--include-unmodelled` | – | optional | also propose fields set in Akeyless but absent from the model |
| `--propose-removals` | – | optional | propose removing objects that are in the repository but not in Akeyless |

**Examples.**

```
scripts/watchover reconcile --repo-url https://ghe.example.com/platform/akeyless-governance.git --profile
↪  akl-readonly
scripts/watchover reconcile --repo-dir ~/src/akeyless-governance --ref main --transport api
scripts/watchover reconcile --repo-dir . --export-dir export/ --out /tmp/reconcile
scripts/watchover reconcile --repo-url git@ghe.example.com:platform/akeyless-governance.git --profile ro --only
↪  '^/governance/roles/payments/' --propose-removals
```

**Sample run.** The stand-in GitHub Enterprise repository has one role changed in the console, one role created there and one role only in Git: (Shown as you would run it; executed against the local stand-in.)

```
$ scripts/watchover reconcile --repo-url https://ghe.example.com/platform/akeyless-governance.git --profile akl-readonly --out
↪  reconcile/; sed -n '20,31p' reconcile/REPORT.md
exported (model): 3 roles, 7 auth-methods, 1 folders, 1 custom-fields, 1 key-policies; 8 item(s) in REPORT.md
reconcile: 1 changed, 1 only in Akeyless, 1 only in the repository; 2 file(s) in the proposal
review reconcile/REPORT.md and reconcile/changes.patch - nothing was committed or pushed
## Changed outside Git (1)

### roles `/governance/roles/payments/team-admin`  - `terraform/model/roles/payments.yaml`
- **rules** changed:
  - only in the repository: `{"capabilities": ["list", "read"], "path": "/payments/+/nonprod/*"}`
  - only in Akeyless: `{"capabilities": ["create", "list", "read"], "path": "/payments/+/nonprod/*"}`

## Only in Akeyless (1)

### roles `/governance/roles/platform/god`  - `terraform/model/roles/imported.yaml`
- created outside Git: proposed as an addition + import block in `imports.tf`

                                                                                                    exit code 1
```

## 4.6 `forwarder`

**Synopsis** `scripts/watchover forwarder [options]` *offline*

**Does** Scaffold a new event forwarder (model entry or HCL); --import-existing adopts one by name.

**How**

- Validates name, type, event types, sources and runner with the same rules as the Terraform guards.
- Builds the model entry; secrets become `*_var` keys of TF_VAR_event_forwarder_secrets.
- Prints YAML (default) or HCL; --write inserts the entry into governance.yaml without touching the rest of the file and refuses duplicates; --import-existing adds an import block (ID = name).
- Presets: certificates = certificate-pending-expiration, certificate-expired, certificate-provisioning-success, certificate-provisioning-failure; auth-methods = auth-method-pending-expiration, auth-method-expired, multi-auth-failure, uid-rotation-failure; rotation = next-automatic-rotation, rotated-secret-success, rotated-secret-failure, rotation-usage; secrets = dynamic-secret-failure, static-secret-updated, secret-sync; access-requests = request-access, apply-justification, email-auth-method-approved; operations = gateway-inactive, rate-limiting, usage, usage-report; critical = certificate-expired, certificate-provisioning-failure, auth-method-expired, rotated-secret-failure, dynamic-secret-failure, multi-auth-failure, uid-rotation-failure, gateway-inactive.

**Exit codes** 0 written; 2 invalid input or duplicate (nothing written)

| Option | Value | Required / default | Description |
|---|---|---|---|
| `--name` | `NAME` | **required** | forwarder name, e.g. /governance/events/secops-mail |
| `--type` | `email\|webhook\|slack\|teams\|service_now` | **required** | forwarder type |
| `--description` | `DESCRIPTION` | optional | description |
| `--preset` | `access-requests\|auth-methods\|certificates\|critical\|operations\|rotation\|secrets` | *repeatable* | add a group of event types (repeatable); the events of each preset are listed under 'How it works' |

| Option | Value | Required / default | Description |
|---|---|---|---|
| `--event` | `EVENT` | *repeatable* | add one event type (repeatable) |
| `--items-source` | `ITEMS_SOURCE` | *repeatable* | item path pattern to watch, e.g. '/pki/*' (repeatable) |
| `--targets-source` | `TARGETS_SOURCE` | *repeatable* | target path pattern to watch (repeatable) |
| `--auth-method-source` | `AUTH_METHOD_SOURCE` | *repeatable* | auth method path pattern to watch (repeatable) |
| `--gateway-source` | `GATEWAY_SOURCE` | *repeatable* | Gateway cluster URL to watch (repeatable; required for teams) |
| `--runner` | `immediate\|periodic` | default immediate | send at once or as a periodic digest |
| `--every` | `EVERY` | optional | hours between digests (required with --runner periodic) |
| `--disabled` | – | optional | create it disabled |
| `--key` | `KEY` | optional | protection key for the forwarder's secret (default: account key) |
| `--email-to` | `EMAIL_TO` | *repeatable* | email: recipient (repeatable) |
| `--include-error` | – | optional | email: include error details |
| `--override-url` | `OVERRIDE_URL` | optional | email: Gateway URL used in links (port 18888) |
| `--url` | `URL` | optional | webhook: URL (slack/teams URLs are secrets and go in the secrets map) |
| `--auth-type` | `AUTH_TYPE` | optional | webhook: user-pass/bearer-token/certificate; service_now: user-pass/jwt |
| `--username` | `USERNAME` | optional | webhook: user for user-pass |
| `--host` | `HOST` | optional | service_now: instance host |
| `--admin-name` | `ADMIN_NAME` | optional | service_now: admin name (user-pass) |
| `--client-id` | `CLIENT_ID` | optional | service_now: client ID (jwt) |
| `--user-email` | `USER_EMAIL` | optional | service_now: user email (jwt) |
| `--format` | `yaml\|hcl` | default yaml | model entry (yaml) or plain resource (hcl) |
| `--write` | `GOVERNANCE_YAML` | optional | yaml: insert the entry into this governance.yaml |
| `--import-existing` | – | optional | the forwarder already exists in Akeyless: also print the import block (import ID = name) |

### Examples.

```
scripts/watchover forwarder --name /governance/events/secops-mail --type email --email-to secops@example.com
↪  --preset critical --items-source '/*' --auth-method-source '/*'
scripts/watchover forwarder --name /governance/events/pki-slack --type slack --preset certificates --items-source
↪  '/pki/*' --write terraform/model/governance.yaml
scripts/watchover forwarder --name /governance/events/siem --type webhook --url https://siem.example.com/akeyless
↪  --auth-type bearer-token --preset critical --auth-method-source '/*' --format hcl
scripts/watchover forwarder --name /governance/events/legacy-mail --type email --email-to ops@example.com --event
↪  certificate-expired --items-source '/*' --import-existing
```

### Sample run.

```
$ scripts/watchover forwarder --name /governance/events/secops-mail --type email --email-to secops@example.com --preset critical
↪  --items-source '/*'
# add under event_forwarders: in terraform/model/governance.yaml
  /governance/events/secops-mail:
    type: email
    event_types:
    - certificate-expired
    - certificate-provisioning-failure
    - auth-method-expired
    - rotated-secret-failure
    - dynamic-secret-failure
    - multi-auth-failure
    - uid-rotation-failure
    - gateway-inactive
[... 5 lines omitted ...]
```

exit code 0

## 4.7 conjur

**Synopsis** `scripts/watchover conjur [options]` *offline*

**Does** Translate Conjur policy into model roles + identity bindings.

**How**
- Parses the policy files (!policy, !variable, !host, !layer, !group, !user, !permit, !grant), resolving ids against their policy namespace.
- Maps privileges to capabilities (execute -> read, read -> list, update -> update) on the migrated item paths; one role per layer or group; nested grants are flattened.
- Writes roles/conjur.yaml, bindings.yaml (host/user -> roles, auth method TODO) and REPORT.md.

**Exit codes** 0 written; 2 unreadable or invalid policy

| Option | Value | Required / default | Description |
|---|---|---|---|
| `policy` | `POLICY...` | **required** | Conjur policy YAML files (loaded at the root namespace in the given order) |
| `--item-prefix` | `ITEM_PREFIX` | default /conjur | Akeyless folder the Gateway migration imports into |
| `--role-prefix` | `ROLE_PREFIX` | default /governance/roles/ | prefix of the generated Akeyless role names |
| `--out` | `OUT` | **required** | output directory (roles/conjur.yaml, bindings.yaml, REPORT.md) |

**Examples.**

```
scripts/watchover conjur root.yml prod.yml --item-prefix /conjur --out export/conjur
```

**Sample run.**

```
$ scripts/watchover conjur scripts/tests/fixtures/conjur.yml --out export/conjur && sed -n '1,9p' export/conjur/roles/conjur.yaml
2 roles, 2 identities, 1 note(s) -> export/conjur
roles:
  /governance/roles/conjur/group-payments-admins:
    allow_external_auth_methods: true
    description: Translated from Conjur group 'payments-admins'
    rules:
    - capabilities:
      - list
      - update
      path: /conjur/prod/payments/db/password
```

exit code 0

## 4.8 account-settings

**Synopsis** `scripts/watchover account-settings [options]` *live*

**Does** Apply account_settings_cli (settings the provider cannot manage) with the CLI.

**How**
- Reads account_settings_cli from governance.yaml; rejects unknown flags and keys Terraform manages.
- Builds one `akeyless update-account-settings` call (lists repeat the flag, booleans as =true/false); --dry-run prints it, otherwise runs it with --profile or --token.

**Exit codes** 0 applied/printed; 2 invalid settings; else the CLI's exit code

**Environment** `AKEYLESS_TOKEN`: token used when --token is not given

| Option | Value | Required / default | Description |
|---|---|---|---|
| `--governance` | `GOVERNANCE` | **required** | path to model/governance.yaml (reads account_settings_cli) |
| `--profile` | `PROFILE` | optional | akeyless CLI profile of an account admin |
| `--token` | `TOKEN` | optional | Akeyless token (default: $AKEYLESS_TOKEN, e.g. from 'watchover token') |
| `--akeyless` | `AKEYLESS` | default akeyless | path to the akeyless CLI binary |
| `--dry-run` | – | optional | print the update-account-settings command, do not run it |

**Examples.**

```
scripts/watchover account-settings --governance terraform/model/governance.yaml --dry-run
AKEYLESS_TOKEN=$(scripts/watchover token) scripts/watchover account-settings --governance
↪  terraform/model/governance.yaml
```

**Sample run.**

```
$ scripts/watchover account-settings --governance terraform/model/governance.yaml --dry-run
+ akeyless update-account-settings --allowed-client-type=cli --allowed-client-type=ui --allowed-client-type=sdk
↪  --bound-ips=10.0.0.0/8 --lock-allowed-client-type=true --lock-bound-ips=true --max-rotation-interval=90
↪  --max-rotation-interval-enable=true --usage-event-enable=true --usage-event-interval=90 --usage-event-object-type=item
                                                                                                    exit code 0
```

## 4.9 `token`

**Synopsis** `scripts/watchover token [options]` *live*

**Does** CI: exchange the pipeline JWT for an Akeyless token.

**How**
- POSTs {access-id, access-type: jwt, jwt} to <api>/auth and prints the token. No credential is stored.

**Exit codes** 0 token printed; 1 missing settings or authentication failed

**Environment** `AKEYLESS_API`: API base URL
`AKEYLESS_ACCESS_ID`: CI auth method access ID
`CI_JWT`: CI-issued JWT

| Option | Value | Required / default | Description |
|---|---|---|---|
| `--api` | `API` | optional | Akeyless API (default: $AKEYLESS_API or https://api.akeyless.io) |
| `--access-id` | `ACCESS_ID` | optional | access ID of the CI auth method (default: $AKEYLESS_ACCESS_ID) |
| `--jwt` | `JWT` | optional | the CI-issued JWT (default: $CI_JWT; prefer the env var - arguments are visible in the process list) |

**Examples.**

```
CI_JWT=$JWT AKEYLESS_ACCESS_ID=p-xxxx scripts/watchover token
```

**Sample run.** (Shown as you would run it; executed against the local stand-in.)

```
$ CI_JWT=$CI_JWT AKEYLESS_ACCESS_ID=p-ci0demo scripts/watchover token
t-demo-0b3f9c
                                                                                                    exit code 0
```

## 4.10 `build-skill`

**Synopsis** `scripts/watchover build-skill [options]` *offline*

**Does** Build the agent-neutral skill package (Agent Skills ZIP) and the prompt pack.

**How**
- Copies SKILL.md (+ agents/ sidecar), docs/*.md as references/ and the kit as template/; validates the result; zips it.
- Writes the prompt pack: full and short system prompt + knowledge files.

**Exit codes** 0 written; 2 built skill invalid

| Option | Value | Required / default | Description |
|---|---|---|---|
| `--out` | `OUT` | optional | output directory, replaced (default: dist/ in the kit root) |

**Examples.**

```
scripts/watchover build-skill
scripts/watchover build-skill --out /tmp/wo-dist
```

**Sample run.**

```
$ scripts/watchover build-skill
built dist/akeyless-watchover.zip (207 KiB)
built dist/akeyless-watchover-prompt-pack.zip (63 KiB)
skill folder: dist/akeyless-watchover/  prompt pack: dist/prompt-pack/
```

exit code 0

## 4.11 `install-skill`

**Synopsis** `scripts/watchover install-skill [options]` *offline*

**Does** Install the skill where an AI agent looks for skills (per-agent folders or any folder).

**How**
- Builds the package in a temporary folder and copies it to the agent's skills folder (project or user scope, or --dest). Never overwrites without --force.

**Exit codes** 0 installed; 2 target exists (no --force) or custom without --dest

| Option | Value | Required / default | Description |
|---|---|---|---|
| `--agent` | `agents\|claude\|codex\|copilot\|cursor\|gemini\|custom` | **required** | target agent; 'agents' = the shared .agents/skills folder; 'custom' needs --dest |
| `--scope` | `project\|user` | default project | project = inside --project; user = your home directory |
| `--project` | `PROJECT` | default . | repository the project-scope skill is installed into |
| `--dest` | `DEST` | optional | explicit skills folder (overrides --agent/--scope) |
| `--force` | – | optional | replace an existing installation |
| `--dry-run` | – | optional | print the target folder, change nothing |

**Examples.**

```
scripts/watchover install-skill --agent codex --project ../akeyless-governance
scripts/watchover install-skill --agent claude --scope user
scripts/watchover install-skill --agent copilot --project . --dry-run
scripts/watchover install-skill --agent custom --dest ~/my-agent/skills --force
```

**Sample run.**

```
$ mkdir -p ../governance-repo && scripts/watchover install-skill --agent codex --project ../governance-repo
installed akeyless-watchover for OpenAI Codex (CLI, IDE, app) in <tmp>/governance-repo/.agents/skills/akeyless-watchover

                                                                                                    exit code 0
```

## 4.12 `validate-skill`

**Synopsis** `scripts/watchover validate-skill [options]` *offline*

**Does** Check a skill folder against the Agent Skills specification.

**How**
- Checks SKILL.md: only specification fields; name 1-64 chars, lowercase/digits/hyphens, equal to the folder; description <= 1024; compatibility <= 500; metadata string -> string; body <= 500 lines; every references/*.md it mentions exists.

**Exit codes** 0 valid; 1 problems listed

| Option | Value | Required / default | Description |
|---|---|---|---|
| `path` | `PATH` | **required** | skill folder (default: this skill) |

**Examples.**

```
scripts/watchover validate-skill
scripts/watchover validate-skill dist/akeyless-watchover
```

**Sample run.**

```
$ scripts/watchover validate-skill
ok    skill/akeyless-watchover follows the Agent Skills specification

                                                                                                    exit code 0
```

## 4.13 `mcp`

**Synopsis** `scripts/watchover mcp [options]` *offline*

**Does** Serve the kit, the docs and the skill commands to any MCP client (stdio).

**How**
- JSON-RPC 2.0 over stdin/stdout. Tools are generated from each command's options and run `scripts/watchover <command>` in the kit root; output and exit code are returned.
- Resources: SKILL.md, docs, model files. Prompts: one per skill command (skill instructions + request).
- Commands that call Akeyless are listed only with --live.

**Exit codes** runs until the client closes stdin

**Environment** `WATCHOVER_MCP_LIVE`: 1 = also expose the commands that call Akeyless (verify, export, account-settings, token)
`WATCHOVER_REPO`: kit root the tools run in (default: this repository)

| Option | Value | Required / default | Description |
|---|---|---|---|
| `--repo` | `REPO` | optional | kit root the tools run in and the model/docs are read from (default: $WATCHOVER_REPO or this repository) |
| `--live` | – | optional | also expose the commands that call Akeyless (same as WATCHOVER_MCP_LIVE=1) |

| Option | Value | Required / default | Description |
|---|---|---|---|
| `--list` | – | optional | print the tools, resources and prompts, then exit |

**Examples.**

```
scripts/watchover mcp
WATCHOVER_MCP_LIVE=1 scripts/watchover mcp --repo ~/src/akeyless-governance
```

**Sample run.**

```
$ scripts/watchover mcp --list | cut -c1-150
tools:     watchover_help, watchover_test, watchover_simulate, watchover_forwarder, watchover_conjur, watchover_build_skill,
↪  watchover_validate_skill
resources: watchover://skill/SKILL.md, watchover://docs/README.md, watchover://docs/agents.md, watchover://docs/api-vs-cli.md,
↪  watchover://docs/archit
prompts:   watchover-help, watchover-design, watchover-change, watchover-review, watchover-pipeline, watchover-import,
↪  watchover-reconcile, watchover-
```

exit code 0

## 4.14 `help` and `version`

**Synopsis** `scripts/watchover help [topic] [--markdown]`; `scripts/watchover version` *offline*

**Does** Prints the overview, one command's entry, the skill commands, or this reference as Markdown; `version` prints the version. No argument = `help`.

| Option | Value | Required / default | Description |
|---|---|---|---|
| `topic` | `TOPIC` | **required** | a kit command name or 'skill'; omitted = overview |
| `--markdown` | – | optional | print this reference as Markdown (docs/usage-guide.md) |

**Sample run.**

```
$ scripts/watchover help
watchover 1.7.0 - akeyless-watchover kit

Usage: scripts/watchover <command> [options]

Commands:
  test              Run every offline check (what CI runs).
  simulate          Offline: evaluate access_expectations.yaml against the model.
  verify            Live: compare expectations with describe-permissions run as each persona's CLI profile.  [needs Akeyless]
  export            Export existing config and translate it: YAML model + import blocks, or plain Terraform (--hcl).  [needs
  ↪  Akeyless]
  reconcile         Compare the governance repository (e.g. GitHub Enterprise) with live Akeyless; write a report and a patch to
  ↪  review. Read-only: never commits or pushes.  [needs Akeyless]
  forwarder         Scaffold a new event forwarder (model entry or HCL); --import-existing adopts one by name.
  conjur            Translate Conjur policy into model roles + identity bindings.
[... 13 lines omitted ...]
```

exit code 0

# A Model files

All under `terraform/model/`. Every key: `docs/model-reference.md`. Secret values are never in the model: keys ending in `_var` name entries of the sensitive `*_secrets` variables.

| File | Top-level keys | Defines |
|---|---|---|
| `roles/<bu>.yaml` | `roles` | access roles: `rules` (path, capabilities, type), `associations` (auth method, sub-claims), management-plane attributes, `allow_*` flags |
| `auth_methods.yaml` | `auth_methods` | auth methods by type (oidc, saml, oauth2, aws_iam, azure_ad, k8s, universal_identity, api_key) and their bindings |
| `folders.yaml` | `folders` | explicit item folders (description, delete protection) |
| `governance.yaml` | `custom_fields`, `key_policies`, `account_settings`, `account_settings_cli`, `event_forwarders` | account policy; `account_settings_cli` is applied by `account-settings`, `event_forwarders` written by `forwarder` |
| `pki.yaml` | `ca_keys`, `issuers` | private CA keys and certificate issuers |
| `gateway.yaml` | `allowed_access`, `log_forwarding`, `migrations` | Gateway admin plane (needs `gateway_api`) |
| `access_expectations.yaml` | `personas`, `checks` | expected access, read by `simulate` and `verify` |

# B Guard messages

Preconditions that stop the plan before anything reaches Akeyless (`terraform_data.model_guards`).

| Message starts with | Fix |
|---|---|
| Duplicate role name | keep the role in one `roles/*.yaml` file |
| Object names must be absolute | leading `/`, no trailing `/` |
| Role / Auth method names must start with | follow `var.naming`, or set the prefix to `""` |
| Unknown auth method type | use a modelled type; others via `export --hcl` |
| Unsupported rule type | item-, target-, role-, auth-method-, sra-rule (`ara-rule` needs `allow_ara_rules`) |
| Invalid capability for the rule type | use the capabilities listed in the message |
| A deny rule must contain only 'deny' | split deny and allow into two rules |
| approval_authority cannot be combined | approvers in a separate rule or role |
| Root-scope rule | narrow the path, or `allow_root_scope: true` with a reason |
| Invalid management-plane value | see `docs/model-reference.md` |
| Association references an auth method that is not in | model it, or `allow_external_auth_methods: true` |
| Auth method has force_sub_claims = true | add `sub_claims` to every association |
| PKI issuer: … | one of signer key / CA target; `auto_renew` needs `destination_path`; CRL/OCSP/ACME/public CA need `gw_cluster_url` |
| custom_fields.object_type | STATIC_SECRET, DYNAMIC_SECRET or ROTATED_SECRET |
| event_forwarders: type must be / runner_type | valid type and events; `every` for periodic; `url_var`; teams needs |
| periodic / slack and teams need url_var / of type teams require | gateway sources and runner – `forwarder` checks all of these |
| model/gateway.yaml defines gateway objects | set `gateway_api` |
| gateway allowed_access / log_forwarding.type | valid permission; exactly one of auth_method/access_id; syslog or splunk |

# C Variables

## C.1 Environment

| Variable | Used by |
|---|---|
| `AKEYLESS_API` | `token`, `export --transport api`: API URL (default `https://api.akeyless.io`) |
| `AKEYLESS_ACCESS_ID` | `token`: CI auth method access ID; with `AKEYLESS_ACCESS_KEY`: provider API-key login |
| `CI_JWT` | `token`: the CI-issued JWT |
| `AKEYLESS_TOKEN` | `export --transport api`, `account-settings`: token when `--token` is absent |
| `TF_VAR_pipeline_jwt`, `TF_VAR_pipeline_access_id` | Terraform login (CI) |
| `TF_VAR_*_secrets` | JSON maps for `*_var` keys: auth methods, event forwarders, gateway |
| `WATCHOVER_REPO`, `WATCHOVER_MCP_LIVE` | `mcp`: kit root; `1` exposes live commands |

## C.2 Terraform input variables

**`terraform/.`**

| Variable | Default | Purpose |
|---|---|---|
| `model_dir` | `"model"` | Directory (relative to the root module) holding the YAML model. |
| `auth_method_secrets` | {} *(sensitive)* | Secrets referenced by *_var keys in model/auth_methods.yaml (e.g. OIDC client secrets). Inject from CI, never commit. |
| `event_forwarder_secrets` | {} *(sensitive)* | Secrets referenced by *_var keys in model/governance.yaml event_forwarders (webhook passwords/tokens/certificates, Slack/Teams URLs, ServiceNow credentials). |
| `gateway_secrets` | {} *(sensitive)* | Secrets referenced by *_var keys in model/gateway.yaml (e.g. Splunk token). |
| `naming` | {} | Naming policy enforced by the guards. Set a prefix to "" to disable that check (null falls back to the default). |
| `allow_ara_rules` | `false` | Allow rule type ara-rule. Not documented by provider 2.0.5 - enable only after verifying on your account. |
| `akeyless_api` | `"https://api.akeyless.io"` | Akeyless API endpoint for control-plane objects: https://api.akeyless.io (default) or a Gateway V2 API. |
| `gateway_api` | `null` | Gateway V2 API (https://<gw>:8000/api/v2 or https://<gw>:8081) used for gateway_* resources. null = gateway objects disabled. One Gateway cluster per state/workspace. |
| `pipeline_jwt` | **required** *(sensitive)* | CI OIDC/JWT token (export TF_VAR_pipeline_jwt in the pipeline). |
| `pipeline_access_id` | **required** | Access ID of the CI/CD JWT (OAuth2) auth method created by ../bootstrap. |

**`bootstrap/.`**

| Variable | Default | Purpose |
|---|---|---|
| `akeyless_api` | `"https://api.akeyless.io"` | – |
| `ci` | **required** | CI system trusted to run Terraform. |
| `managed_scopes` | {} | Paths the apply identity may administer. Keep '/*' unless the account hosts objects that must stay out of reach of this pipeline. |

# D Files

```
terraform/   generic HCL (versions, main = loading + guards, auth_methods, roles, governance, pki,
             gateway, outputs), model/ (YAML), tests/ (mocked-provider tests, fixtures, guard proofs)
bootstrap/   one-time CI identity: OAuth2/JWT auth method + plan/apply roles (own state)
scripts/     watchover + the tools it runs; tests/
ci/          GitHub Actions and GitLab templates, CODEOWNERS example
skill/       SKILL.md (Agent Skills format) + agents/openai.yaml sidecar
docs/        topic documents (bundled in the skill as references/), this reference (manual/)
dist/        build-skill output: akeyless-watchover/ (+ .zip), prompt-pack/ (+ .zip)
```

**Sample runs.** The sample runs in Chapter 4 were executed by `docs/manual/build.py` when this PDF was built, in a copy of the repository. Live commands ran against a stand-in that answers from the test fixtures (`docs/manual/demo/`); the `verify` stand-in contains one console change, to show a mismatch. Long outputs are cut (`[... N lines omitted ...]`); temporary paths are shown as `.` or `<tmp>`.

**Further reading.** `docs/`: `rbac-model.md` (authorization rules), `path-and-role-design.md`, `model-reference.md`, `terraform.md`, `pipeline.md`, `cli.md`, `api-vs-cli.md`, `testing-and-verification.md`, `brownfield-import.md`, `conjur-migration.md`, `pki.md`, `agents.md`, `reconcile.md`, `limitations.md` (VERIFY items).